CO1035: Game Concepts

Lab 2B: Drawing Sprites

---

1. Introduction

- In the previous labs, we learned how to place models, manipulate models, use custom cameras, understand the fundamentals of user input, and how textures can be used in our games
- This lab focuses on demonstrating how to display text and use sprites in your TL-Engine projects.

---

2. Setting Up

a. Create a new TL-Engine project called **Lab2B_Text_Project**.

b. Create a **kManual camera** at position (0, 0, 0), then **implement camera control using keyboard and mouse** as you did in **Camera Control Project**.

c. Before utilizing text in the TL-Engine, you need to **load a font**. Add the code below in the setup area of your code to load a font:

```
// Load "Comic Sans MS" font at 36 points
IFont* myFont = myEngine->LoadFont( "Comic Sans MS", 36 );
```

As you can see, fonts are loaded in a similar way to meshes. A new data type, IFont, is used for the fonts. As with meshes, you need both the data type of the variable and a name for the variable.

d. Once a font has been loaded, you can draw text with it. However, the text will only be drawn for one frame. To keep the text permanent, you must put the code for drawing the text **in the game loop** (in the while loop). This should display "Hello World" on your screen.

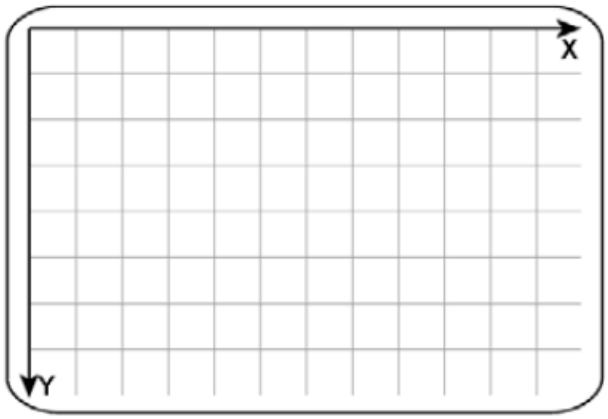
```
myFont->Draw("Hello World", 200, 100);
```

e. You can load any system font, e.g., **Times New Roman, Adobe Garamond Pro, Comic Sans MS, Monotype Corsiva, stencil** std, etc. **Not every font is guaranteed to work correctly**. Experiment and see what happens. If none is loaded, the font defaults to 24-point Arial.

f. Now **load** the cube mesh and **create** a model from it.

g. Make sure you have **implemented camera movement** similar to the one in **Camera Control Project** and use it to control your camera. Once you have done this, move the camera around. What did you notice?

h. Text (and Sprites, as you'll see later on) use a different coordinate system from the one used to draw 3D models. **Text uses the X and Y axes of the screen.** This alternative coordinate system is the same as the one used by the mouse. The screen axes look like this:



The origin (0, 0) of the coordinate system is at the top left of the screen. Y increases as you go down the screen, whilst x increases as you go to the right.

---

3. Positioning Text

a. Unlike a model, you cannot move text directly. Instead, you will have to keep track of where you would like the text to be drawn.

b. **Declare two floating-point variables** called **fontX** and **fontY**. Set them both initially to a value of 0.

c. Draw the text using **fontX** and **fontY**:

```
myFont->Draw("Hello World", fontX, fontY);
```

d. **Set up the cursor keys** so that they change the values of fontX and fontY within the game loop and so **move the text**. The cursor keys are: **Key_Up, Key_Down, Key_Right** and **Key_Left**. You should then be able to move the location of the text around the screen.

e. There are several other things that you can do with the 'Draw' as shown in the examples below:

```
// This will draw red text, centre aligned at the coordinates (100, 300)
myFont->Draw( "Hello Again", 100, 320, kRed, kCentre );
//The will draw the phrase in blue in exactly the centre of the screen.
myFont->Draw( "Hello Again", 640, 320, kBlue, kCentre, kVCentre );
```

Note: The coordinates used in the last Draw method are based on the default window size of TLEngine projects (1280, 720). If you are using a different window size, you should take half of your custom dimensions when calculating coordinates for the center of your screen. You could also utilize the TL-Engine's **GetWidth()** and **GetHeight()** methods. For the definition of the **Draw()** method, see the appendix at the end of this document.

f. The color of the text is specified in the fourth parameter of the method. The following standard colors are supported: **kWhite**, **kGreen**, **kMagenta**, **kDarkGrey**, **kBlack**, **kBlue**, **kYellow**, **kRed**, **kCyan**, **kLightGrey**.

g. The alignment of the text can be specified in the fifth (horizontal) and sixth (vertical) parameters. Horizontal alignment refers to the horizontal positioning of the text in relation to the coordinates specified. By default, the text starts at the given coordinates (i.e., it is left-aligned). However, the text can be centered at the coordinates or right-aligned. The constants for horizontal alignment are: **kLeft**, **kRight**, **kCentre**.

Vertical alignment refers to the vertical positioning of the text in relation to the coordinates specified. The constants for vertical alignment are: **kTop**, **kVCentre**, **kBottom**.

Using these constants, try to position your text in the top left, top right, bottom left, bottom right, and absolute center of the screen.

---

4. Displaying More Complex Text

a. Create a new TL-Engine project called **Lab2B_KeepingScore_Project** and copy the code from the **Lab2B_main_activity_lab_code.cpp** file into the project's .cpp file (check the path to the default media folder to avoid errors). The file can be found on Blackboard.

b. Run the project, and you will see a simple game where you press the space bar to shoot a sphere at moving cubes. **Get familiar with the gameplay and the code**; you will be adding display messages to this game in this section.

c. To output complex text (including variables or functions/methods), you need to be introduced to a new part of C++. You may have used "**cout**" in this module. However, it is also possible to use the '**stringstream**' type, which can be used like '**cout**' to create a string. The **stringstream** type is used by tL-Engine to display complex text. The steps involved are as follows:

- Include the sstream library in your program

```
#include <sstream>
using namespace std;
```

- Create a variable of type **stringstream** to hold the text you want to draw on the screen. Declare the variable within the game loop; otherwise you'll get problems with repeating text.

```
stringstream outText;
```

- Assign to it the text you wish to display (variables can be assigned), for example:

```
outText << "Sphere Position: " << sphere->GetX();
```

- **Use the draw method** to draw the text within the **stringstream** variable; make sure you use the str() command in conjunction with **stringstream** variable to convert its data into a string of characters to be drawn:

```
myFont ->Draw( outText.str(), 20, 20 );
outText.str(""); // Clear myStream
```

d. Although **stringstream** gives you access to functionality similar to "**cout**" do note that outputting newlines (**endl** or '\n') has no effect in the TL-Engine - use more than one 'Draw' call instead.

e. Back to your **Lab2B_KeepingScore_project**. Your first task is to write code that displays the value of "score", the variable that keeps track of the number of cubes hit by the player. Take a moment to first think about where this code should be placed.

f. Include the **stringstream** library and remember that you need to use the std namespace as well.

g. **Declare** an **IFont** variable called **regularFont** and **load** a font of your choice with a font size of 36.

h. Declare a **stringstream** variable within the game loop and assign to it the value of score (using the format - Score: 0). Then draw it using regularFont at the top right corner of your screen. Make sure you use the appropriate coordinates, align constants, and colour.

i. Currently, the game stops when it enters either gamewon or gameover states. Remove the lines of code that stops the game in each of these states and replace them with code that draws an appropriate message at the center of the screen. Display "Game Over" and the current score when the game is in gameover state, and display "You Won!!!" when the game is in gamewon state.

j. Create a new **IFont** variable called **largeFont** and load the same font type as **regularFont** and a font size of 72. Use **largeFont** to display the game over and game won messages.

---

5. Drawing Sprites

a. Sprites are 2D bitmaps. Sprites were once used to draw all of the graphics in games. However, with the introduction of 3D graphics, they are no longer the center of computer graphics. Whilst models have overtaken sprites as the mainstay of computer graphics, sprites are still extremely important. Sprites are commonly used within **User Interfaces, Menus and Loading Screens**. Sprites can be used to create effects such as smoke, explosions and rain. In this section, you will use a sprite as a **backdrop for a User Interface**. In many ways, you will find the sprite operations similar to those of models. However, sprites are simpler in that you only have to consider movement in 2D: up and down, or left and right. **The sprites do have a Z component**, but this is only used to sort the sprites, i.e., to calculate which sprite is in front, and hence visible, if two or more sprites overlap. The naming convention follows the format you are familiar with. Essentially, you can replace the word Model with the word Sprite. **An important difference is that you do not need to load a Mesh**.

b. The data type for a sprite pointer is **ISprite**. Sprites are created using **CreateSprite()**. Formally, **CreateSprite()** is a method of **myEngine**. There is a set of movement and positioning methods for sprites. These are: **GetX**, **GetY**, **SetX**, **SetY**, **SetZ**, **SetPosition**, **MoveX**, **MoveY**, and **Move**. All pixel coordinates are in **floating point**.

```
ISprite* backdrop = myEngine->CreateSprite("backdrop.jpg", 200, 400);
backdrop->moveX(0.1f);
```

c. Create a new project called **Lab2B_Sprites_Project**.

d. Load "BlackBall.jpg" as a sprite backdrop and position it at the bottom of your screen. Then draw the phrase "User Interface" over the sprite backdrop in white.

e. The techniques described above can be used to output real-time information from your game. Load the mesh for a sphere and create one sphere. Set up key controls in order to move the sphere. Use the GetX, GetY and GetZ methods and the text output technique you have just learnt to output the current X, Y, and Z coordinates of the sphere as you move it around. The values should be drawn in the User Interface area.

6. Appendix

a. **Draw()** method definition:

```
Draw (const string& sText, const int iX, const int iY, const unsigned int uiColour, const
EHorizAlignment eHorizAlign, const EVertAlignment eVertAlign)
```